

efreilogo.png

Rapport — Optimisation Convexe

Sujet A : Comparaison des optimiseurs sur CIFAR-10
SGD, SGD+Momentum, Adam, RMSProp, AdaGrad

[Thi Tho PHAM] [Prénom NOM]
[Prénom NOM] [Prénom NOM]

EFREI Paris — Cours d'Optimisation Convexe

Juin 2026

Résumé

Ce rapport présente une étude comparative des principaux algorithmes d'optimisation utilisés pour l'entraînement de réseaux de neurones convolutifs sur le jeu de données CIFAR-10. Nous analysons le comportement de cinq optimiseurs — SGD, SGD avec momentum, Adam, RMSProp et AdaGrad — selon plusieurs critères : vitesse de convergence, précision finale, généralisation et robustesse aux hyperparamètres. Une attention particulière est portée à la nature non convexe du paysage de perte des réseaux profonds, et à la façon dont chaque optimiseur y répond.

Table des matières

1	Introduction	3
2	Fondamentals des CNNs et des optimiseurs	3
2.1	Réseaux de neurones convolutifs et apprentissage profond	3
2.2	Descente de gradient stochastique (SGD)	3

2.3	SGD avec momentum	3
2.4	AdaGrad	4
2.5	RMSProp	4
2.6	Adam	4
2.7	Non-convexité et implications pratiques	4
3	État de l’art sur CIFAR-10	5
3.1	Résultats de référence	5
3.2	Comparaisons d’optimiseurs dans la littérature	5
4	Protocole expérimental	5
4.1	Architecture	5
4.2	Données et prétraitement	5
4.3	Optimiseurs et hyperparamètres	6
4.4	Métriques et reproductibilité	6
5	Résultats et analyse	6
5.1	Courbes de convergence	6
5.2	Précision finale et vitesse de convergence	6
5.3	Sensibilité au taux d’apprentissage	6
5.4	Normes des gradients	6
6	Discussion	7
6.1	Lien avec la non-convexité	7
6.2	Adam vs SGD : le paradoxe de la généralisation	7
6.3	Limites de l’étude	7
7	Conclusion	7

1 Introduction

Dans ce projet, nous choisissons d'utiliser un réseau de neurones convolutif (CNN) pour résoudre cette tâche de classification. Les CNNs se sont imposés comme l'architecture de référence pour la vision par ordinateur grâce à leur capacité à extraire automatiquement des caractéristiques visuelles hiérarchiques — des contours et textures aux formes complexes — directement depuis les pixels bruts. Notre objectif n'est pas de maximiser la précision du modèle, mais d'utiliser ce cadre pour observer et comparer le comportement de cinq optimiseurs : SGD, SGD avec momentum, AdaGrad, RMSProp et Adam.

2 Fondementals des CNNs et des optimiseurs

2.1 Réseaux de neurones convolutifs et apprentissage profond

L'apprentissage profond (*deep learning*) s'agit l'entraînement de réseaux de neurones à plusieurs couches, capables d'apprendre automatiquement des représentations hiérarchiques à partir des données brutes. Les CNNs est un l'architecture de référence pour la vision par ordinateur.

Un CNN est composé de trois types de couches : les couches de convolution qui extraient des motifs visuels locaux, les couches de pooling qui réduisent la dimension spatiale, et les couches fully-connected qui produisent la décision finale de classification.

L'entraînement des réseaux revient à minimiser une fonction de coût $\mathcal{L}(\theta)$ non convexe. Nous comparons cinq algorithmes : SGD, SGD avec momentum, AdaGrad, RMSProp et Adam

2.2 Descente de gradient stochastique (SGD)

La descente de gradient stochastique, pour chaque itération, les paramètres θ sont mis à jour dans la direction opposée au gradient de la fonction de coût :

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t) \quad (1)$$

où $\eta > 0$ est le taux d'apprentissage et $\nabla_{\theta} \mathcal{L}$ est le gradient estimé sur un mini-batch. La convergence peut être lente lorsque la surface de perte présente des courbures très différentes selon les directions.

2.3 SGD avec momentum

Le SGD avec momentum utilise un vecteur de vitesse pour la mise à jour de base, ce qui permet de conserver une direction de descente cohérente d'une itération à l'autre :

$$v_{t+1} = \mu v_t + \eta \nabla_{\theta} \mathcal{L}(\theta_t) \quad (2)$$

$$\theta_{t+1} = \theta_t - v_{t+1} \quad (3)$$

Le coefficient $\mu \in [0, 1)$ (typiquement 0,9) contrôle la contribution des gradients passés. Cette mémoire accélère la convergence dans les directions de gradient stable.

2.4 AdaGrad

AdaGrad [6] introduit l'idée d'adapter automatiquement le taux d'apprentissage à chaque paramètre, en le divisant par la racine de la somme cumulée des carrés des gradients passés :

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} g_{t,i} \quad (4)$$

Les paramètres qui reçoivent de grands gradients voient leur taux d'apprentissage diminuer rapidement, tandis que les paramètres peu mis à jour conservent un taux élevé.

2.5 RMSProp

RMSProp [7] reprend l'idée d'AdaGrad mais remplace la somme cumulative par une moyenne mobile exponentielle, ce qui évite que le taux d'apprentissage ne s'annule avec le temps :

$$v_t = \rho v_{t-1} + (1 - \rho) g_t^2 \quad (5)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t + \epsilon}} g_t \quad (6)$$

avec $\rho \approx 0,9$. Seuls les gradients récents influencent la mise à l'échelle, ce qui rend RMSProp plus stable et efficace pour l'entraînement de réseaux profonds.

2.6 Adam

Adam (Adaptive Moment Estimation) [5] combine les idées du momentum (premier moment) et de l'adaptivité de RMSProp (second moment) :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (7)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (8)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t + \epsilon}} m_t \quad (9)$$

avec $\beta_1 = 0,9$, $\beta_2 = 0,999$ et $\epsilon = 10^{-8}$ par défaut. Adam ajoute une correction de biais sur m_t et v_t pour les premières itérations. Sa robustesse au choix du taux d'apprentissage en fait un optimiseur pratique et largement adopté.

2.7 Non-convexité et implications pratiques

Les fonctions de coût des réseaux profonds sont non convexes : elles présentent des minima locaux, des points-selles et des plateaux qui compliquent l'optimisation. En pratique, les optimiseurs basés sur le gradient ne garantissent pas de trouver le minimum global, mais convergent vers des solutions acceptables grâce au bruit stochastique et à la haute dimensionnalité de l'espace des paramètres. Le choix de l'optimiseur influence non seulement la vitesse de convergence, mais aussi la qualité des minima trouvés et donc la capacité du modèle à généraliser.

3 État de l’art sur CIFAR-10

3.1 Résultats de référence

Le tableau 1 résume les performances publiées sur CIFAR-10 pour des architectures comparables.

TABLE 1 – Précision sur CIFAR-10 — état de l’art (sélection)

Modèle	Optimiseur	Précision (%)	Référence
ResNet-20	SGD + momentum	91.2	[2]
VGG-16	SGD + momentum	93.6	[3]
ResNet-56	SGD + momentum	93.0	[2]
DenseNet-100	SGD + momentum	95.5	[4]

3.2 Comparaisons d’optimiseurs dans la littérature

Wilson et al. [8] montrent empiriquement que les méthodes adaptatives (Adam, RMSProp) convergent plus vite mais généralisent moins bien que SGD avec momentum sur les tâches de vision. Ils attribuent cet écart à la géométrie des minima trouvés. Cette observation constitue un point central de notre analyse. Recander de toujours termine par utilise Adam a la fin chaque model

4 Protocole expérimental

4.1 Architecture

Nous utilisons un réseau convolutif à [X] couches (à décrire selon votre implémentation), entraîné avec les mêmes hyperparamètres fixes pour tous les optimiseurs (sauf le taux d’apprentissage, optimisé par grille pour chaque méthode).

4.2 Données et prétraitement

CIFAR-10 contient 50 000 images d’entraînement et 10 000 images de test, réparties équitablement sur 10 classes (avion, automobile, oiseau, chat, cerf, chien, grenouille, cheval, bateau, camion). Les augmentations appliquées sont : retournement horizontal aléatoire, recadrage aléatoire (padding=4), normalisation par la moyenne et l’écart-type du jeu d’entraînement.

4.3 Optimiseurs et hyperparamètres

TABLE 2 – Configuration des optimiseurs

Optimiseur	Paramètres	Taux d'apprentissage
SGD	–	[valeur]
SGD + Momentum	$\mu = 0.9$	[valeur]
AdaGrad	$\epsilon = 10^{-8}$	[valeur]
RMSProp	$\rho = 0.9, \epsilon = 10^{-8}$	[valeur]
Adam	$\beta_1 = 0.9, \beta_2 = 0.999$	[valeur]

4.4 Métriques et reproductibilité

Toutes les expériences sont reproduites avec $[N]$ graines aléatoires différentes. Nous mesurons la précision sur le jeu de validation (10% des données d'entraînement) et sur le jeu de test, ainsi que l'écart de généralisation (précision entraînement – précision test).

5 Résultats et analyse

5.1 Courbes de convergence

[Insérer ici la figure des courbes de perte et de précision par optimiseur]

FIGURE 1 – Courbes d'entraînement et de validation pour les cinq optimiseurs.

5.2 Précision finale et vitesse de convergence

TABLE 3 – Résultats finaux après $[N]$ epochs

Optimiseur	Précision train	Précision test	Écart génér.	Epochs à 80%
SGD	–	–	–	–
SGD + Momentum	–	–	–	–
AdaGrad	–	–	–	–
RMSProp	–	–	–	–
Adam	–	–	–	–

5.3 Sensibilité au taux d'apprentissage

[Insérer figure ou tableau montrant l'effet du lr pour chaque optimiseur]

5.4 Normes des gradients

[Insérer figure de l'évolution de la norme $\|\nabla_{\theta}\mathcal{L}\|$ au cours de l'entraînement]

6 Discussion

6.1 Lien avec la non-convexité

Les résultats obtenus illustrent les difficultés propres à l'optimisation non convexe. [Discuter ici en lien avec les points-selles, les minima plats vs étroits, l'effet du bruit stochastique].

6.2 Adam vs SGD : le paradoxe de la généralisation

Conformément aux observations de [8], nous observons que [préciser vos résultats]. Cela s'explique par la tendance d'Adam à converger vers des minima plus étroits [?], ce qui dégrade la généralisation malgré une meilleure convergence en entraînement.

6.3 Limites de l'étude

[Taille du modèle, budget de calcul, choix des hyperparamètres, nombre de seeds, etc.]

7 Conclusion

Ce travail a mis en évidence les différences de comportement des optimiseurs classiques sur un problème de classification non convexe. [Résumer les 2-3 conclusions principales]. Ces résultats rappellent que le choix de l'optimiseur ne se réduit pas à sa vitesse de convergence : la géométrie du paysage de perte et la capacité à généraliser sont des critères tout aussi déterminants.

Références

- [1] Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images*. Technical Report, University of Toronto.
- [2] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 770–778.
- [3] Simonyan, K., & Zisserman, A. (2014). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. International Conference on Learning Representations (ICLR).
- [4] Huang, G., Liu, Z., Van der Maaten, L., & Weinberger, K. Q. (2017). *Densely Connected Convolutional Networks*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 4700–4708.
- [5] Kingma, D. P., & Ba, J. (2015). *Adam : A Method for Stochastic Optimization*. International Conference on Learning Representations (ICLR).
- [6] Duchi, J., Hazan, E., & Singer, Y. (2011). *Adaptive Subgradient Methods for Online Learning and Stochastic Optimization*. Journal of Machine Learning Research, 12, 2121–2159.
- [7] Tieleman, T., & Hinton, G. (2012). *Lecture 6.5 — RMSProp : Divide the gradient by a running average of its recent magnitude*. COURSERA : Neural Networks for Machine Learning.
- [8] Wilson, A. C., Roelofs, R., Stern, M., Srebro, N., & Recht, B. (2017). *The Marginal Value of Momentum for Small Learning Rate SGD*. International Conference on Learning Representations (ICLR).